CHARUSAT
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF INFORMATION TECHNOLOGY

A.Y. 2026-27 Semester-3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical-6

Practical AIM:

To implement stack using array and linked list, and apply stack-based logic to expression conversion, evaluation, and bracket validation problems.

6.1

Problem Defination:

A cafeteria stacks clean trays on a fixed-size counter. New trays are always placed on top, and customers always take from the top. The counter can hold at most n trays at a time — if it is full, no more trays can be added, and if it is empty, no tray can be taken. Given a sequence of place and take operations, implement this fixed-capacity tray stack and print the current top tray after each operation. Report an error if a place or take operation cannot be performed. Describe how your solution tracks whether the stack is full or empty. What are the exact conditions you check before a push and before a pop, and what does your program do when those conditions fail?

Output:

```
Enter stack size: 2
Enter number of operations: 2

1. Place tray (Push)
2. Take tray (Pop)
Enter choice: 1
Enter tray number:2
Top tray:2

1. Place tray (Push)
2. Take tray (Pop)
Enter choice: 2
Removed tray:2
Stack is empty.
```

Difficulty Faced:

The main difficulty was handling the full and empty stack conditions correctly.

Solution:

I used the `top` variable to check whether the stack is full or empty before performing push and pop operations.

Outcome:

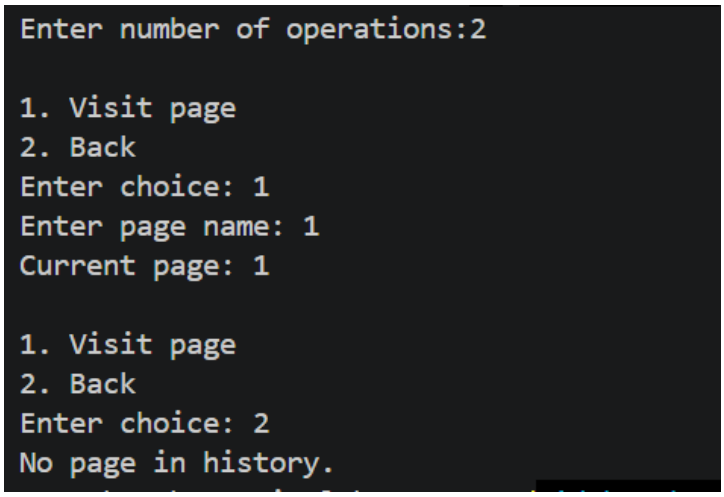
The fixed-capacity stack was successfully implemented with proper overflow and underflow handling.

6.2

Problem Defination:

A web browser keeps track of pages visited so that the back button always returns to the most recently visited page. Unlike a fixed counter, the browser has no hard limit on how many pages it can remember — it grows as the user visits more pages and shrinks as they press back. Given a sequence of visit and back operations, implement this unlimited page history and print the current page after each operation. Describe how your solution differs from Problem 1 in how it handles memory. What happens in your solution when the user presses back on the very first page with no history left?

Output:



Difficulty Faced:

The main difficulty was managing the page history without a fixed size and handling the back operation when no previous page is available.

Solution:

I used a dynamic stack that grows when new pages are visited and shrinks when the back button is pressed. Unlike Problem 1, there is no fixed capacity. Before performing back, the program checks whether the stack is empty.

Outcome:

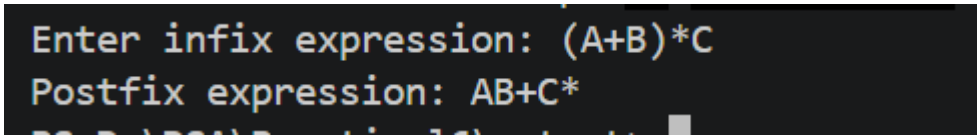
The browser history was successfully implemented with dynamic memory management. If the user presses back on the first page, the program displays “No history available” and keeps the current page unchanged.

6.3

Problem Defination:

A calculator application receives arithmetic expressions typed by users in the usual way — numbers and operators written in between, with brackets to control priority (e.g. 3 + 4 * 2 or (3 + 4) * 2). Internally the calculator needs to convert each expression into a form where operators appear after their operands, so it can evaluate them without needing to re-read brackets. Given an infix expression, convert it to the equivalent postfix form and print the result. Describe the role the stack plays in your conversion. What does your solution do when it encounters a closing bracket, and what happens to the operators that were waiting on the stack at that point?

Output:



Difficulty Faced:

The main difficulty was handling operator precedence and brackets correctly while converting the expression.

Solution:

I used a stack to temporarily store operators and brackets. When a closing bracket was encountered, operators were popped until the matching opening bracket was found.

Outcome:

The infix expression was successfully converted into the correct postfix form while maintaining the required operator order.

Key Questions:

1.A circular array is one way to reuse freed slots without shifting all elements. Without implementing it, explain what change would need to be made to your front and rear pointer logic to make the array behave circularly, and what new condition would indicate a full array.

Sol: In a circular array, the front and rear pointers should wrap around to the beginning when they reach the last index. This can be done using (index + 1) % n. The array is considered full when (rear + 1) % n == front.

2.Your linked list stack has no fixed capacity limit. But in practice, can it still run out of space? What determines the actual limit, and why is that limit invisible in your code?

Sol: Yes, a linked-list stack can still run out of space. Its actual limit depends on the available memory (RAM/heap) of the system. This limit is not fixed in the program because nodes are created dynamically as needed. When the system cannot allocate memory for a new node, the stack cannot grow further.

3.operator precedence determines the order in which operators are placed into the output. Without tracing any expression, explain what would go wrong in your output if your solution treated all operators as having equal precedence.

Sol: If all operators are treated as having equal precedence, the postfix output may place operators in the wrong order. For example, multiplication should be performed before addition, but ignoring precedence could make addition come first, producing an incorrect result.

Github Link:

https://github.com/brindajivani/DSA/tree/main/Practical6